Taller de Criptografía Aplicada

SecureChat: comunicación segura con criptografía híbrida

Diseño e implementación de un chat cifrado entre dos procesos locales

Modalidad: individual o en parejas.

Lenguaje: Python 3.

Comunicación: sockets TCP/IP sobre `localhost`.

Biblioteca criptográfica sugerida: `cryptography`.

Algoritmos sugeridos: RSA (mínimo 2048 bits, con OAEP) para el establecimiento de la sesión y AES-256-GCM para proteger los mensajes.

Entrega principal: código fuente, breve documentación y video de demostración.

Diseño: el enunciado define propiedades y resultados esperados; no prescribe nombres de funciones ni una estructura de código específica.

Curso: _____

Estudiante(s): _____

Fecha: _____

1. Objetivos del taller

1.1. Objetivo general

Construir una aplicación de consola capaz de mantener una conversación bidireccional entre dos procesos que se ejecutan en la misma computadora, utilizando **criptografía asimétrica durante el establecimiento de la sesión** y **criptografía simétrica autenticada para proteger cada mensaje de la conversación**.

1.2. Objetivos específicos

Al finalizar el taller, el estudiante deberá ser capaz de:

- Diferenciar el papel de la criptografía simétrica y la criptografía asimétrica dentro de un mismo sistema.
- Establecer una sesión segura sin transmitir una clave simétrica en texto plano.
- Proteger mensajes en tránsito de forma que el contenido legible no viaje por el socket.
- Utilizar cifrado autenticado para detectar modificaciones del ciphertext.
- Explicar qué información es pública, qué información debe permanecer secreta y qué datos viajan realmente por la red.
- Demostrar experimentalmente el flujo completo: plaintext local → ciphertext en tránsito → plaintext recuperado.

2. Conceptos esenciales

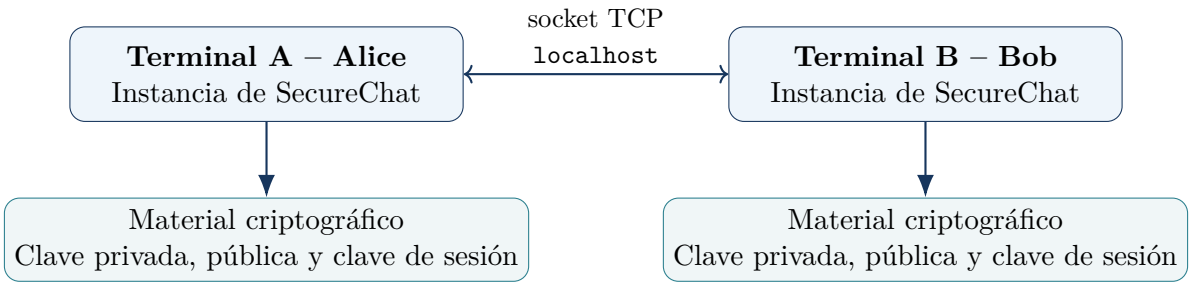
Plaintext	Mensaje legible antes de ser protegido criptográficamente.
Ciphertext	Representación cifrada que puede transmitirse por una red sin revelar directamente el contenido original.
Clave pública / privada	Par de claves relacionado matemáticamente. La pública puede compartirse; la privada debe permanecer bajo control exclusivo de su propietario.
Handshake	Intercambio inicial mediante el cual los participantes establecen las condiciones criptográficas de una sesión segura.
Clave de sesión	Secreto simétrico utilizado durante una conversación concreta para cifrar y autenticar mensajes de manera eficiente.
Criptografía híbrida	Uso conjunto de criptografía asimétrica para establecer/proteger secretos y criptografía simétrica para proteger grandes cantidades de datos.
Nonce	Valor que debe ser único para cada cifrado bajo una misma clave cuando se utiliza AES-GCM. No es una contraseña ni necesita mantenerse secreto.
Authentication tag	Valor de autenticación que permite detectar si un ciphertext fue modificado antes de ser aceptado por el receptor.

Idea central del taller

La criptografía asimétrica resuelve el problema inicial de establecer de forma segura el material de sesión. Una vez establecida la sesión, la criptografía simétrica protege la conversación porque es más eficiente para transmitir múltiples mensajes.

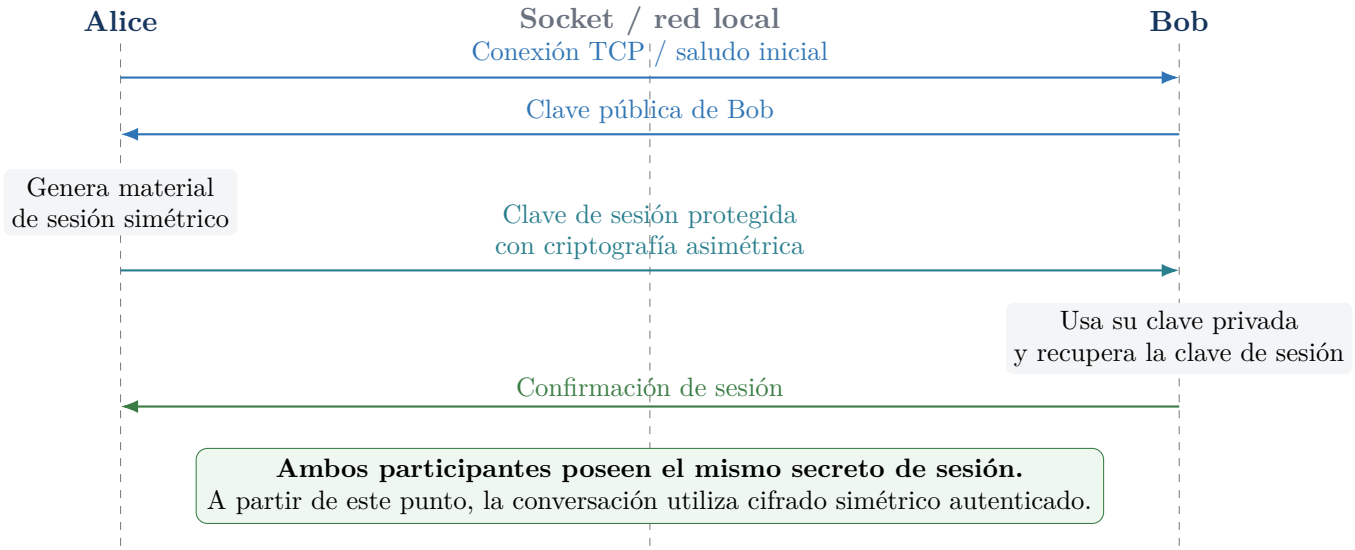
3. Arquitectura esperada

El mismo proyecto deberá ejecutarse dos veces, una instancia por participante. Ambas instancias se comunican mediante una conexión TCP real sobre la interfaz local de la computadora.



3.1. Secuencia 1: establecimiento de la sesión

El siguiente diagrama representa el comportamiento mínimo esperado. La implementación exacta queda a criterio del estudiante; no se prescribe una estructura específica de funciones.



La clave simétrica de sesión no puede enviarse en texto plano. Las claves privadas no pueden transmitirse por el socket. La información binaria puede serializarse para su transporte, pero codificar en Base64 o hexadecimal **no equivale a cifrar**.

4. Desarrollo del taller

Paso 1 — Preparar la comunicación entre dos procesos

Implemente una aplicación que pueda ejecutarse en dos terminales de la misma computadora. Una instancia debe poder esperar conexiones y la otra iniciar la conexión mediante TCP/IP sobre `localhost`.

Debe cumplirse:

- Ambos procesos permanecen activos simultáneamente.
- Cada participante puede identificar claramente con quién está conectado.
- La aplicación permite enviar y recibir más de un mensaje durante la misma sesión.

Antes de integrar la criptografía, las dos instancias deben ser capaces de establecer una conexión estable. Esta etapa es únicamente de construcción y no representa el resultado seguro final.

Paso 2 — Preparar el material criptográfico asimétrico

Cada participante deberá disponer de un par de claves asimétricas. La clave privada debe mantenerse local y la clave pública debe poder utilizarse durante el establecimiento de la sesión.

Algoritmo sugerido: RSA de al menos 2048 bits. Para proteger la clave de sesión se sugiere un esquema moderno de padding como OAEP.

Debe cumplirse:

- Las claves privadas nunca viajan por la conexión.
- Las claves públicas pueden intercambiarse o cargarse según el diseño adoptado.
- El programa no debe revelar en pantalla el contenido completo de una clave privada.

El estudiante debe poder explicar qué clave puede compartirse, cuál debe protegerse y por qué la clave pública no reemplaza a la clave privada.

Paso 3 — Realizar el handshake y establecer la clave de sesión

Antes de permitir el chat, ambas instancias deberán completar un handshake que utilice criptografía asimétrica para proteger o establecer el secreto que se empleará en la conversación.
Debe cumplirse:

- La clave de sesión no puede estar escrita de forma fija en el código.
- La clave de sesión no puede transmitirse como plaintext.
- Al finalizar el handshake, ambos extremos deben disponer del material simétrico necesario para proteger la conversación.
- La aplicación debe indicar de forma visible cuándo la sesión segura ha sido establecida.

Alice y Bob terminan el handshake con material de sesión compatible, sin haber expuesto la clave simétrica en texto plano durante el transporte.

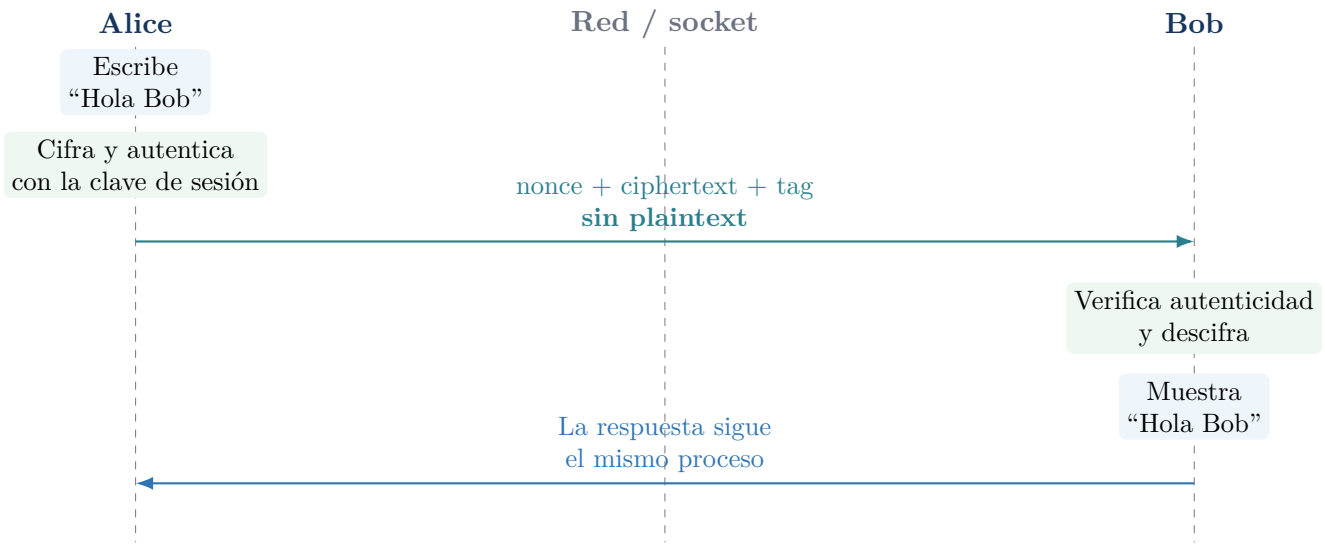
Paso 4 — Proteger cada mensaje con criptografía simétrica autenticada

Una vez completado el handshake, todos los mensajes de usuario deberán protegerse antes de utilizar el socket.
Algoritmo sugerido: AES-256-GCM.
Debe cumplirse:

- Cada mensaje se cifra antes de transmitirse.
- Cada cifrado utiliza un nonce apropiado y no repetido bajo la misma clave.
- El receptor verifica la autenticidad/integridad antes de aceptar el plaintext recuperado.
- Después del handshake, el texto escrito por el usuario no debe aparecer dentro de los bytes enviados por el socket.

El emisor observa el plaintext local y el ciphertext generado; el receptor observa el ciphertext recibido y, únicamente después de una verificación correcta, el plaintext recuperado.

4.1. Secuencia 2: envío de un mensaje protegido



Paso 5 — Implementar la conversación bidireccional

La aplicación final debe funcionar como un chat: cualquiera de los dos participantes puede enviar mensajes y ambos pueden continuar conversando mientras la sesión permanezca activa.

Debe cumplirse:

- El programa distingue visualmente mensajes locales, datos cifrados transmitidos y mensajes recibidos.
- Se deben poder enviar al menos tres mensajes en cada dirección durante la demostración.
- El usuario debe poder finalizar la sesión de forma controlada.

Dos terminales abiertas simultáneamente muestran una conversación completa. La red transporta ciphertext, mientras que cada participante observa localmente el mensaje legible antes de cifrar o después de descifrar.

Paso 6 — Demostrar detección de manipulación

Realice una prueba controlada en la que se modifique al menos un byte del contenido cifrado antes de que llegue al receptor.

Debe cumplirse:

- El receptor debe detectar que el mensaje fue alterado.
- El mensaje modificado debe rechazarse.
- No debe mostrarse un plaintext como si fuera válido cuando falle la autenticación.

El grupo demuestra que cifrar no es suficiente: el mecanismo de autenticación permite detectar una alteración del ciphertext.

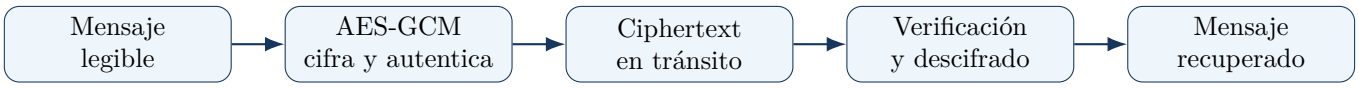
5. Requisitos de seguridad mínimos

La solución será considerada completa únicamente si cumple todos los puntos siguientes:

- ✓ Uso de Python 3 y sockets TCP/IP para la comunicación.
- ✓ Uso de una biblioteca criptográfica reconocida; se sugiere `cryptography`.
- ✓ Handshake basado en criptografía asimétrica.
- ✓ Clave simétrica de sesión generada dinámicamente y no hardcodeada.
- ✓ Ninguna clave privada se transmite por el socket.
- ✓ Mensajes posteriores al handshake cifrados mediante un esquema autenticado; se sugiere AES-256-GCM.
- ✓ Nonce correcto y no repetido bajo la misma clave.
- ✓ Rechazo de mensajes cuyo authentication tag no sea válido.
- ✓ Evidencia visible de plaintext local, ciphertext enviado, ciphertext recibido y plaintext recuperado.
- ✗ No se permite considerar Base64, hexadecimal u otra codificación como sustituto del cifrado.
- ✗ No se permite utilizar ECB para proteger la conversación.

6. Resultado esperado al finalizar

Al concluir el taller debe existir una aplicación funcional que pueda demostrarse con dos terminales en una misma computadora.



La demostración final debe evidenciar, como mínimo:

1. Dos instancias del programa ejecutándose simultáneamente.
2. Handshake completado antes del primer mensaje de usuario.
3. Al menos tres mensajes enviados por Alice y tres mensajes enviados por Bob.
4. Visualización del ciphertext que cruza la conexión.
5. Recuperación correcta del plaintext en el receptor.
6. Una prueba donde un ciphertext alterado sea detectado y rechazado.

7. Entrega

7.1. Código y documentación

Entregue el proyecto completo, incluyendo:

- Código fuente ejecutable.
- Archivo `README` con instrucciones claras para instalar dependencias, iniciar cada instancia y reproducir la demostración.

- Breve descripción del protocolo implementado: algoritmo asimétrico, algoritmo simétrico, tamaño de claves y estructura general de los mensajes transmitidos.

7.2. Video de demostración obligatorio

Grabe un video corto de la pantalla de la computadora (aproximadamente 4–6 minutos) mostrando la aplicación en funcionamiento.

El video debe mostrar claramente:

- Las dos terminales o ventanas de chat visibles en la misma PC.
- El establecimiento de la conexión y la finalización del handshake.
- Una conversación en ambos sentidos.
- Al menos un ejemplo donde se compare el mensaje legible con el ciphertext realmente transmitido.
- La recepción y descifrado del mensaje en el extremo opuesto.
- La prueba de alteración de un ciphertext y el rechazo correspondiente.
- Una explicación breve de dónde se utiliza la criptografía asimétrica y dónde se utiliza la criptografía simétrica.

Trabajo individual o en parejas

El taller puede realizarse individualmente o en parejas. Si se realiza en parejas, ambos integrantes deben comprender la solución y participar en la explicación del video; no es suficiente dividir el código sin comprender el flujo completo del protocolo.

8. Preguntas de cierre

Incluya respuestas breves a las siguientes preguntas en la documentación:

1. ¿Qué problema resuelve la criptografía asimétrica durante el handshake?
2. ¿Por qué la conversación utiliza criptografía simétrica después del handshake?
3. ¿Qué información puede observar un tercero que capture los paquetes de la conversación?
4. ¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?
5. ¿Qué función cumple el authentication tag?
6. ¿Qué ocurriría si la clave de sesión se transmitiera sin protección?
7. ¿Qué ocurriría si un atacante obtiene una de las claves privadas?

Meta conceptual final

El estudiante debe poder explicar cada etapa de la comunicación: cómo se establece la sesión, qué secreto se protege con criptografía asimétrica, qué datos viajan cifrados y por qué el receptor puede detectar una alteración antes de aceptar un mensaje.